МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп’ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №2

з дисципліни «Інженерія даних»

на тему: «PostgreSQL у конвеєрі даних: завантаження, цілісність і транзакції»

Виконав:
Студент 3 курсу
групи ФеП-34
Тимотей Полянський

Перевірив:
ас. Фостяк Маркіян Романович

Львів 2026

1. Мета роботи та стек інструментів

Мета роботи: розгорнути локальний екземпляр PostgreSQL 16 у Docker; спроєктувати тришарову реляційну модель зі схемами `stg`, `core`, `audit`; реалізувати конвеєр потокового завантаження через `COPY`, валідацію бізнес-правил, ізоляцію дефектів у карантин та транзакційну публікацію в `core`; перевірити ідемпотентність і відкат (`ROLLBACK`) при збоях; виконати звірку балансу рядків та аналіз зернистості при `JOIN`.

Конфігурація робочої станції та проєкту:

- **Платформа:** Omarchy (Arch Linux, Wayland), термінал fish, редактор Neovim, менеджер uv.
- **Стек обробки даних:** Python 3.14.7, драйвер psycopg 3 (бінарний протокол COPY), тестовий фреймворк pytest.
- **СУБД:** PostgreSQL 16-alpine (контейнер Docker `cityflow_postgres_s08`), реквізити винесено в `.env` (виключено з Git).
- **Вхідні датасети:** `cityflow_S08_base_v1.zip` та `cityflow_S08_lab02_relational_v1.zip` (SHA-256: `a6f77a3ad9dcffb14a45440406be571b74051f14ed11d2372e62cdfac046a762`).
- **Контроль версій:** Git, lazygit, GitHub CLI. Гілка: `lab/02-postgresql-load`, тег релізу: `lab-02-v1`.
- **Посилання на ресурси:** GitHub / uni_data_engineering | **Pull Request:** PR #2.

2. Реляційна модель та схема просторів імен (DDL)

Архітектура сховища CityFlow реалізована за трирівневою схемою:

1. **`stg` (Staging buffer):** точна копія вхідних CSV, усі бізнес-стовпці типізовані як `TEXT` для виключення збоїв імпорту. Додано метадані: `batch_id`, `source_file`, `source_row_number`, `loaded_at`.
2. **`core` (Domain model):** типізований цільовий шар з природними бізнес-ключами як PK, зовнішніми ключами (`FOREIGN KEY`) та обмеженнями домену (`CHECK`).
3. **`audit` (Operational metadata):** таблиці журналювання запусків (`pipeline_runs`), файлової звірки балансу (`run_file_stats`) та карантину відхилених записів (`rejected_rows`).

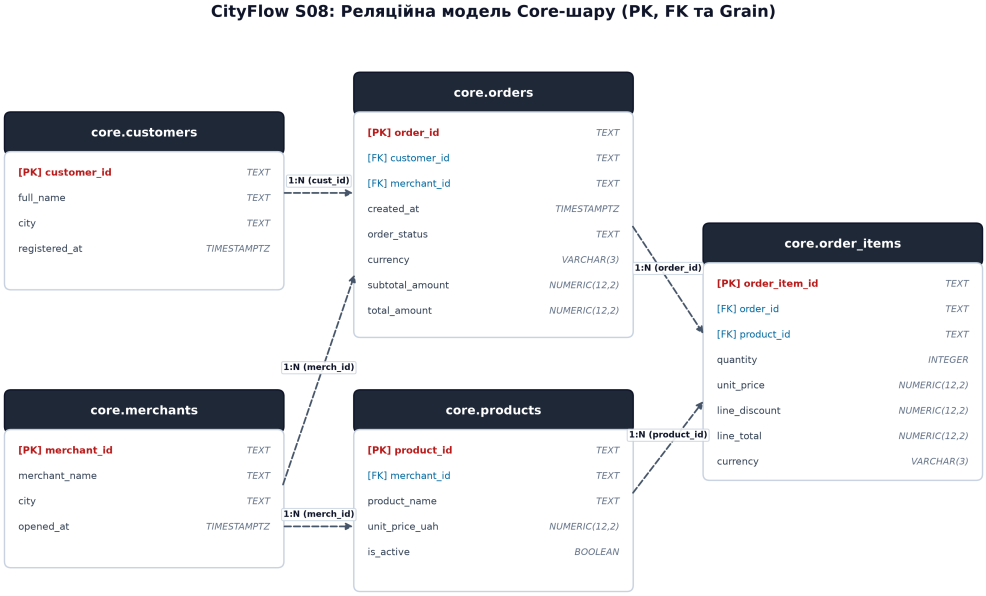


Рисунок 1: ER-діаграма цільового шару core: сутності, первинні та зовнішні ключі

Таблиця core	Первинний ключ	Зернистість (Grain)	Обмеження цілісності
`customers`	`customer_id`	Один зареєстрований клієнт	`full_name, city, registered_at NOT NULL`
`merchants`	`merchant_id`	Один заклад-партнер	`merchant_name, city, opened_at NOT NULL`
`products`	`product_id`	Один товар закладу	`FK merchant_id, CHECK (unit_price_uah > 0)`
`orders`	`order_id`	Одне замовлення сервісу	`FK customer_id, merchant_id, CHECK (total >= 0)`
`order_items`	`order_item_id`	Одна товарна позиція замовлення	`FK order_id, product_id, CHECK (quantity > 0)`

3. Конвеєр завантаження, валідації та карантину

Конвеєр (`src/lab02/pipeline.py`) виконує послідовну обробку даних у три фази:

1. **Потоковий сирий імпорт (`ingest.py`):** потокове зчитування CSV через `psycopg.copy` (`COPY ... FROM STDIN`). Кількість рядків у `stg` строго дорівнює обсягу вхідних файлів (1-в-1).
2. **Ізоляція дефектів (Карантин):** невалідні та дубльовані записи записуються в `audit.rejected_rows` із фіксацією вихідного рядка у `JSONB`, початкового номера рядка CSV та коду причини (`reason_code`). Жоден рядок не видаляється мовчки.
3. **Транзакційна публікація:** перенесення snapshot у `core` виконується в єдиній транзакції (`BEGIN ... COMMIT`) у порядку посилальної цілісності: `customers`, `merchants` -> `products`, `orders` -> `order_items`.

Вхідний файл	Staging	Дублікати	Карантин	Core (Accepted)	Статус балансу
`customers.csv`	801	1	0	800	Збігається (100%)
`merchants.csv`	43	1	0	42	Збігається (100%)
`products.csv`	534	1	2	531	Збігається (100%)
`orders.csv`	6 001	1	3	5 997	Збігається (100%)
`order_items.csv`	14 062	0	33	14 029	Збігається (100%)

Аналіз розподілу відхилень у карантині (`audit.rejected_rows`):

- `FULL_DUPLICATE`: відсіяно по 1 повному дублікату рядка в `customers`, `merchants`, `products`, `orders`.
- `INVALID_PRICE`: 1 товар відхилено через непозитивну ціну (<= 0).
- `ORPHAN_MERCHANT_FK`: 1 товар та 1 замовлення відхилено через посилання на неіснуючого мерчанта.
- `ORPHAN_CUSTOMER_FK`: 1 замовлення відхилено через посилання на неіснуючого клієнта.
- `INVALID_DATE`: 1 замовлення відхилено через синтаксично некоректну мітку часу (`2026-19-45 25:61:00`).
- `PARENT_REJECTED`: 33 позиції `order_items` каскадно ізольовано до карантину, оскільки їхні батьківські замовлення не потрапили в `core.orders`.

4. Контрольні SQL-перевірки цілісності та аналіз Fan-out

За результатами виконання перевірок `sql/lab02/02_checks.sql` зафіксовано такі показники:

1. **Унікальність PK:** кількість дублікатів первинних ключів у всіх 5 таблицях `core` дорівнює **0**.
2. **Посилальна цілісність:** кількість сирітних записів (orphan FK) за всіма зв'язками дорівнює **0**.
3. **Узгодженість агрегатів замовлень та позицій за валютами:**

Валюта	Замовлень	Сума orders (грн)	Позицій	Сума line_total (грн)
EUR	32	354.17	71	305.42
UAH	5 925	3 619 701.14	13 871	3 179 379.65
USD	40	575.10	87	511.46

4. Демонстрація спотворення суми (Join Fan-out) та коректний розрахунок: При прямому об'єднанні таблиць `orders` та `order_items` через `INNER JOIN` виникає множення рядків через зв'язок 1:N. Оскільки замовлення містить декілька позицій, сума `orders.total_amount` дублюється стільки разів, скільки позицій прив'язано до чека:

- Некоректний розрахунок: `SUM(o.total_amount) JOIN order_items` повертає **10 276 248.15 грн** (14 029 рядків).
- Фактична сума замовлень: `SUM(total_amount) FROM core.orders` становить **3 620 630.41 грн** (5 997 рядків).
- **Коректний розрахунок без антипатерну `DISTINCT`:** агрегація позицій проводиться до з'єднання на рівні зернистості замовлення через `CTE`, після чого виконується безпечний `JOIN` 1:1:

```sql
WITH items_agg AS (
    SELECT order_id, sum(line_total) AS order_items_sum
    FROM core.order_items GROUP BY order_id
)
SELECT o.currency, count(o.order_id) AS orders_count,
       sum(o.total_amount) AS total_orders_amount,
       sum(ia.order_items_sum) AS total_items_amount
FROM core.orders o
JOIN items_agg ia ON o.order_id = ia.order_id GROUP BY o.currency;
```

5. Надійність: ідемпотентність та перевірка ROLLBACK

Стійкість конвеєра до збоїв верифікована набором автотестів `pytest` (`tests/lab02/test_pipeline.py`):

- **`test_idempotency_rerun`:** повторна обробка того самого пакета зберігає кількісний склад `core` та суми замовлень незмінними; у журналі `audit.pipeline_runs` створюється новий запис зі статусом `SUCCESS`.
- **`test_controlled_failure_rollback`:** спроба вставки позиції з від'ємною кількістю (`quantity = -1`) порушує обмеження `CHECK(quantity > 0)`. Транзакція публікації повністю відкочується (`ROLLBACK`), стан таблиць `core` залишається неушкодженим, а в журналі зберігається статус `FAILED` з текстом системної помилки.
- **`test_schema_failure_pre_ingest`:** наявність некоректних заголовків CSV аварійно зупиняє роботу до публікації в `core`.

6. Декларація використання ШІ (AI Usage)

Згідно з файлом `AI_USAGE.md`:

- **Інструмент:** LLM-асистент (Gemini).
- **Завдання:** проєктування DDL схем PostgreSQL, реалізація логіки дедуплікації через віконні функції, написання транзакційного завантажувача на `psycopg`, підготовка тестів `pytest` та верстка Typst-шаблону.
- **Спосіб перевірки:** код конвеєра та тести виконано локально в ізольованому середовищі; звірка балансу та контрольні суми підтверджені безпосередніми SQL-запитами до контейнера PostgreSQL 16.

7. Підсумкові висновки

1. Реалізовано стійкий конвеєр завантаження реляційних даних у PostgreSQL 16, що забезпечує 100% аудитоздатність через карантинну схему `audit`.
2. Забезпечено виконання принципу ACID: модифікація цільового шару `core` відбувається виключно в межах неподільної транзакції, що виключає появу напівзавантажених або пошкоджених станів.
3. Доведено неприпустимість прямого розрахунку фінансових сум через зв'язки 1:N без попередньої агрегації замовлень до зернистості первинного ключа.

8. Відповіді на контрольні запитання до захисту

1. Чому для сирого шару (staging) обрано типи TEXT, а типізація відбувається під час перенесення в core?
У джерелах зустрічаються некоректні формати (значення `NaT`, порожні рядки, дати на кшталт `2026-19-45 25:61:00`). Якщо задати суворі типи в `stg`, команда `COPY` перерветься і весь пакет буде втрачено. Текстовий буфер дозволяє прийняти 100% рядків, зафіксувати їхні порядкові номери та безпечно відправити дефекти до карантину.

2. Як забезпечується ідемпотентність завантаження повного зрізу (full snapshot)?
Перед вставкою перевірених рядків цільові таблиці `core` очищуються за допомогою `TRUNCATE ... CASCADE` в межах єдиної транзакції публікації. Багаторазовий запуск на одному й тому самому пакеті не дублює рядки і завжди приводить базу до однакового детермінованого стану.

3. Що таке транзакційна цілісність конвеєра і як реалізовано відкат (ROLLBACK)?
Вона гарантує атомарність (Atomicity з ACID): або всі пов'язані таблиці `core` оновлюються одночасно, або жодна з них. Якщо під час вставки виникає помилка, блок `except` перехоплює виняток і виконує `ROLLBACK`, а статус `FAILED` записується в окремій транзакції.

4. Чому конвеєр не повинен мовчки ігнорувати або видаляти пошкоджені рядки, і як працює карантин?
Мовчазне відсіювання призводить до неконтрольованої втрати даних і викривлення фінзвітності. Карантинна таблиця `audit.rejected_rows` фіксує номер рядка, сирий payload у `JSONB` та код причини (`reason_code`), забезпечуючи повну простежуваність.

5. У якому порядку мають завантажуватися таблиці у core і чому саме в такому?
Порядок диктується зовнішніми ключами (`FOREIGN KEY`): спочатку батьківські таблиці (`customers`, `merchants`), потім сутності другого рівня (`products`, `orders`), і наприкінці — деталізація позицій (`order_items`). Інший порядок викличе помилку порушення FK.

6. Чому виникає завищення суми (fan-out) при JOIN orders та order_items, і як правильно рахувати суму без DISTINCT?
Через зв'язок 1:N одне замовлення дублюється стільки разів, скільки позицій воно містить. Застосування `DISTINCT` є антипатерном, оскільки відкидає різні замовлення з однаковою сумою. Коректний підхід — попередня агрегація `order_items` за `order_id` у `CTE` до виконання `JOIN`.

7. Чим відрізняється поведінка констрейнтів БД від перевірок у коді застосунку?
Перевірки в коді можна випадково обійти (наприклад, через інший клієнт або скрипт). Констрейнти СУБД (`NOT NULL`, `CHECK`, `FK`) вбудовані в ядро рушія бази і гарантують інваріанти даних незалежно від способу доступу.

8. Які переваги дає використання бінарного протоколу та команди COPY у порівнянні з INSERT?
Команда `COPY` передає потік байтів безпосередньо у внутрішній формат збереження PostgreSQL без накладних витрат на індивідуальний синтаксичний аналіз, побудову плану запиту та блокування на кожен рядок, що прискорює завантаження у десятки разів.